

Hippocampal Memory Architecture for AI Agents

A Brain-Inspired Alternative to the Context Window

Technical Architecture Proposal – July 2026

Executive Summary

Current AI agents rely on **context windows** – fixed-size text buffers that concatenate conversation history and feed it to a language model. This is a von Neumann architecture solution to a biological problem: separate storage from processing, copy data between them, truncate when the buffer overflows. The brain never evolved this assumption. It stores patterns in the same neurons that process them, and "retrieval" is reactivation of those neurons via sparse hippocampal pointers.

We propose a **hippocampal memory architecture** that replaces the context window with a brain-inspired alternative: a graph-based hippocampal index over a distributed content store, where working memory is the set of currently activated patterns rather than a fixed text buffer. The system encodes conversation into structured triples, retrieves via graph traversal (pattern completion), consolidates via a graph neural network during offline "dream state" passes, and forgets through managed versioning rather than destructive truncation.

This architecture is built on **WaveDB** – an existing hierarchical key-value database with MVCC concurrency, a Graph layer with Gremlin-style traversal, and production Python bindings – combined with small, specialized models for extraction, reasoning, and consolidation.

1. Problem Statement

1.1 The Context Window Is the Wrong Abstraction

Every major AI assistant – ChatGPT, Claude, Gemini – operates on the same fundamental model:

```
User query → [entire conversation history loaded into context window]
               → LLM processes via self-attention → response
```

This has three structural problems:

Problem Consequence

Fixed capacity When conversation exceeds the token limit, information is permanently lost via truncation, summarization, or sliding windows – all lossy hacks

$O(n^2)$ attention Every token attends to every other token. Computational cost grows quadratically with conversation length

No native memory The model has no persistent memory between sessions. RAG (Retrieval Augmented Generation) adds an explicit retrieval step, but it's bolted on, not integrated

1.2 The Brain Does It Differently

The brain does not "load" memories into a buffer. It stores a **sparse index** in the hippocampus – compressed pointers to the neocortical neurons that, when reactivated together, reconstruct the memory. This is **hippocampal indexing theory** (Teyler & DiScenna, 1986), and it has held up remarkably well across decades of neuroscience.

The retrieval process:

1. **A cue arrives** – a partial input (a face, a smell, a question)
2. **Pattern completion in CA3** – the hippocampal subfield CA3 is a recurrent autoassociative network. It takes the partial cue and fills in the missing components, reconstructing the full hippocampal index. This happens in ~100–200 milliseconds and is entirely unconscious
3. **Neocortical reactivation** – the completed hippocampal index sends signals back to the neocortical sites it points to. Those neurons fire in the same pattern they did during the original experience. The memory is **reconstructed**, not retrieved
4. **The reconstruction IS the short-term access** – there is no separate "working memory copy." Working memory is simply the subset of long-term memory that is currently activated by attention (the **embedded-processes model**, Cowan 1999, 2005)

The brain also has **complementary learning systems**: the hippocampus learns fast and separates patterns, while the neocortex learns slow and extracts statistical regularities. During sleep, the hippocampus replays memories to gradually train the neocortex without catastrophic interference.

1.3 What We're Building

A memory architecture for AI agents that mirrors this design:

Brain Component Our Implementation

Neocortical store (distributed, slow-learning) WaveDB HBTree – hierarchical key-value store with MVCC

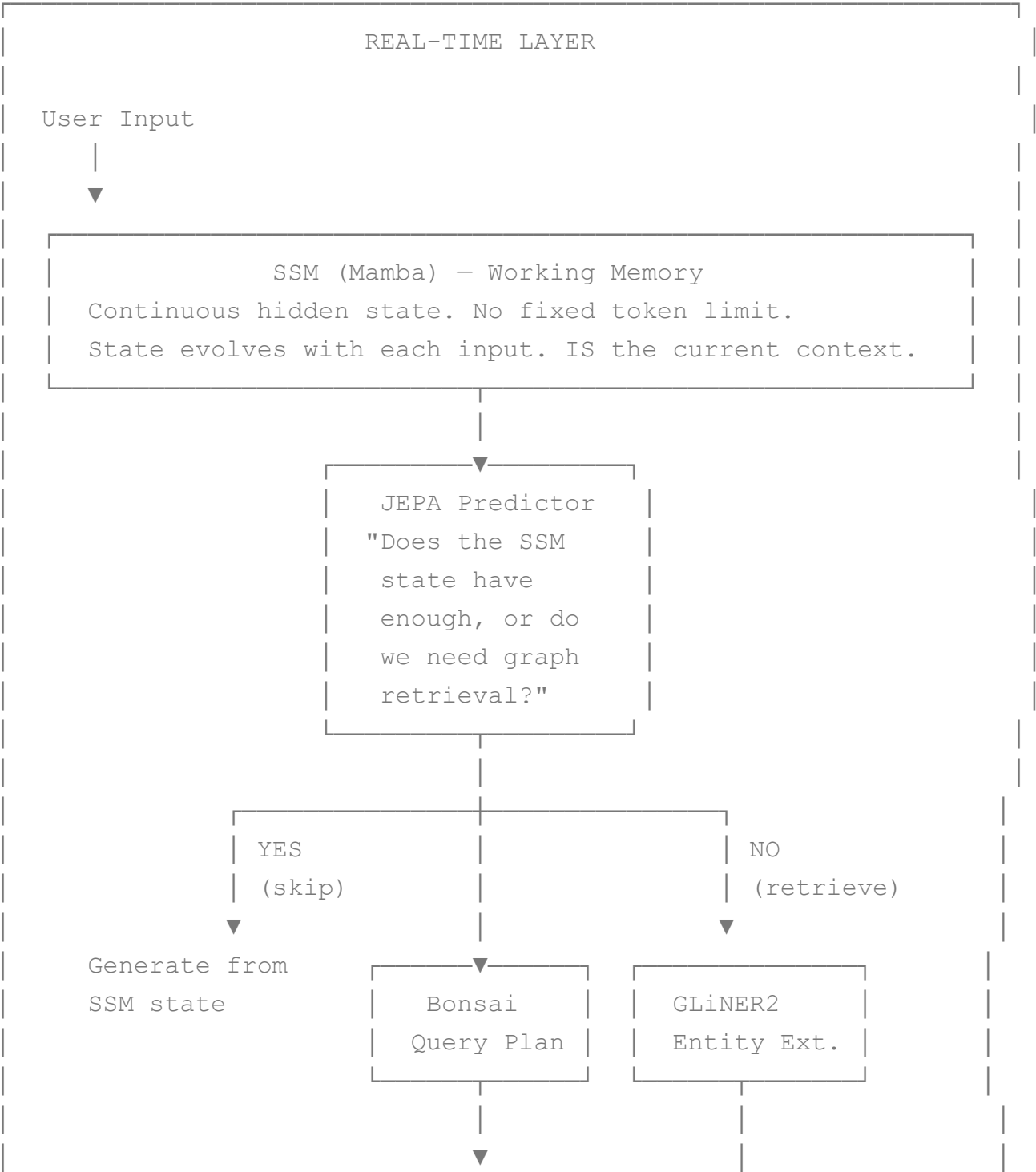
Hippocampal index (sparse, fast-learning) WaveDB Graph layer – triple store with SPO/POS/OSP indexes

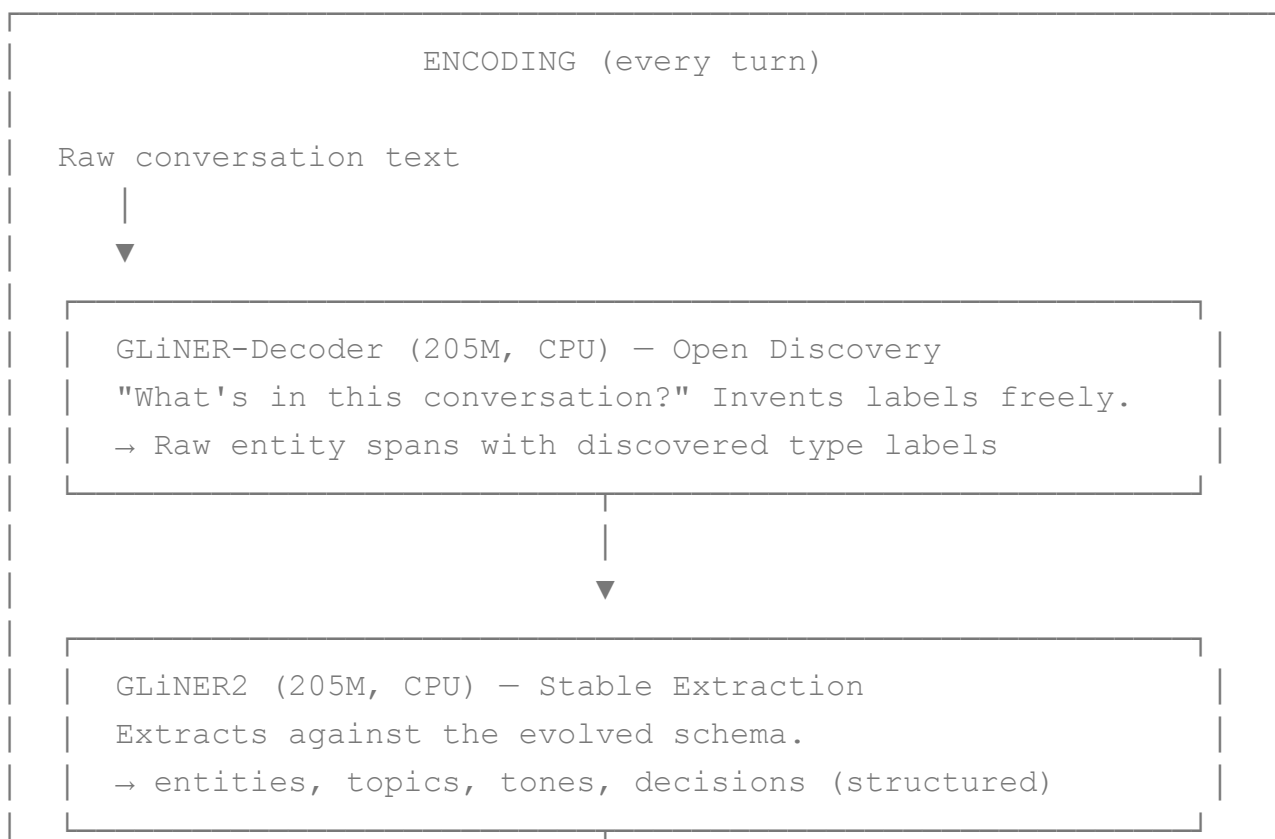
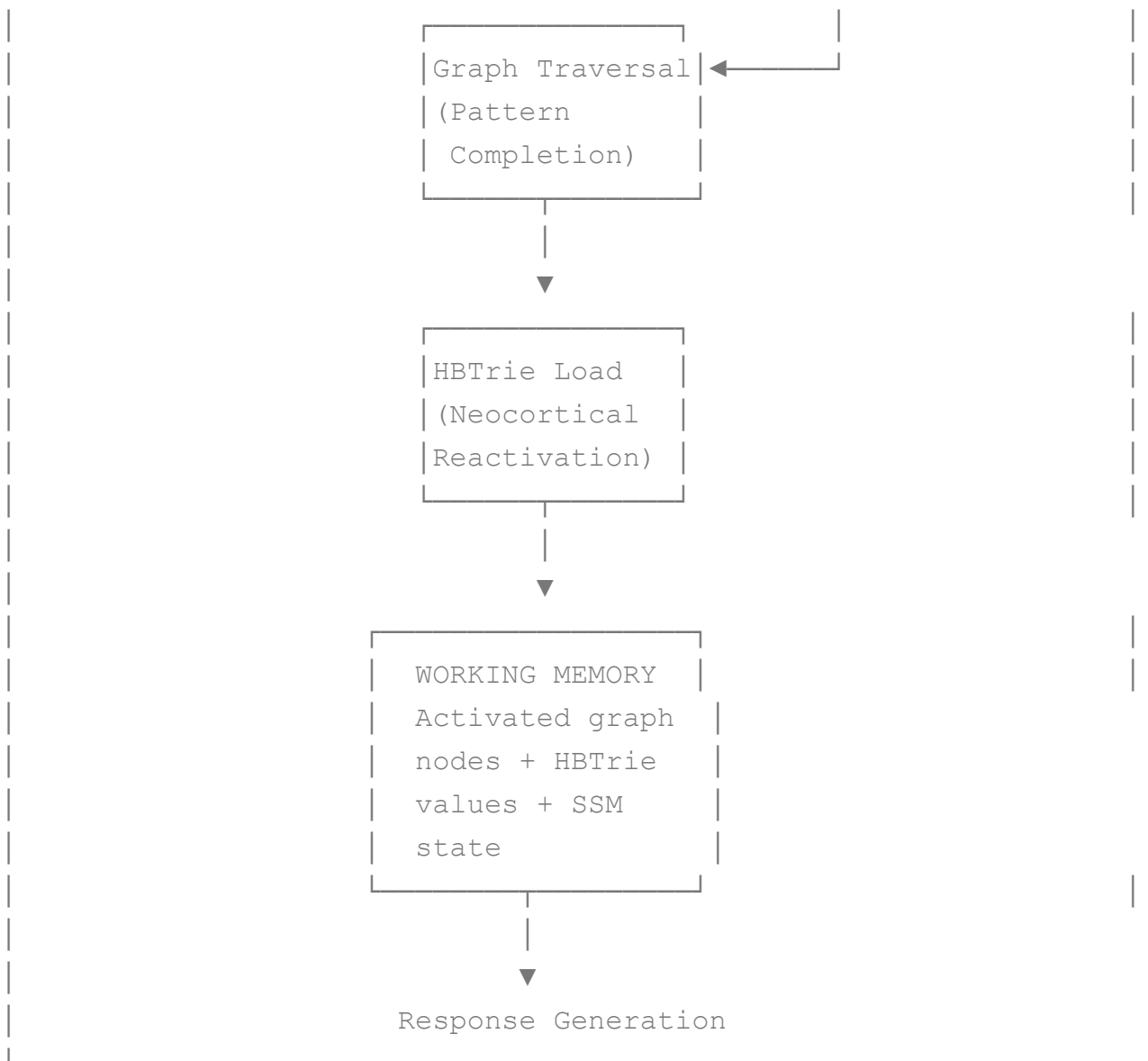
CA3 pattern completion Graph traversal (Gremlin-style) from partial cues

- Sharp-wave ripple replay GNN-driven consolidation during offline "dream state"
 - Working memory SSM (Mamba) continuous hidden state + activated graph nodes
 - Dopamine/salience modulation JEPA predictor + learned salience gate
 - Reality monitoring Bonsai (ternary 8B model) waking during anomalies
 - Systems consolidation Morphism formation + subgraph abstraction into semantic memories
-

2. Architecture Overview

2.1 System Diagram





↓
▼

Bonsai (8B ternary, GPU) – Relation Extraction
"How do these entities relate?"
→ (subject, predicate, object) triples

Graph Layer
(Hippocampal Index)

Triples:
(ep_N, has_entity,
E:Alice)
(ep_N, has_topic,
T:database_design)
(ep_N, has_tone,
A:frustrated)
(ep_N, follows,
ep_N-1)

HBtrie
(Neocortical Store)

Content:
ep/N/summary
ep/N/full_text
ep/N/timestamp

CONSOLIDATION (Dream State – Offline)

GNN Consolidator (~200M, GPU)

❶ Saliency Scoring

- Which nodes/edges are structurally important?
- Learned importance, not just access frequency

❷ Subgraph Summarization (DiffPool)

- Collapse related episodes into semantic memories
- "WaveDB development with Alice (June 20-24)"

❸ Link Prediction (GAE/SEAL)

- Discover implicit edges not explicitly recorded
- (Postgres, related_to, performance) [confidence 0.82]

④ Anomaly Detection

- Flag subgraphs that don't fit expected patterns
- Wake Bonsai for verification

⑤ Ontology Refinement

- Predict missing subClassOf edges
- Detect inconsistent category assignments



Bonsai (wakes on anomalies)

Ternary {-1, 0, +1} judgments:

"Is this predicted edge valid?"

"Is this anomaly real or a false positive?"

"Should these episodes be consolidated?"

Forgetting & Pruning

- Edge utility scores decay with disuse
- Validity windows mark superseded facts
- Reconsolidation: old facts versioned, not deleted
- MVCC preserves all versions for historical queries

GESTATIONAL TRAINING (Oracle-Driven)

Oracle (DeepSeek, large)

Provides training signal for all learned components:

- "GLiNER2 missed 'use_hbtrie' as a decision"
- "Bonsai should have used entity_mode='union' here"
- "GNN salience for ep_003 should be 0.88, not 0.45"
- "JEPA should predict retrieval probability 0.85"

Generates initial training data for:

- Query planning pairs (5K-10K examples)
- Relation extraction examples
- Salience labels for GNN training
- Ontology refinement decisions

2.2 Component Summary

Component Parameters Hardware Role Runs When

WaveDB Graph Layer N/A CPU/disk Hippocampal index — sparse pointers to neocortical patterns

Always

WaveDB HBTrie N/A CPU/disk Neocortical store — actual content (text, embeddings, structured data)

Always

Vector Index (FAISS/Usearch) N/A CPU/GPU Semantic similarity search On retrieval

GLiNER-Decoder 205M CPU Open entity discovery — invents labels freely, discovers new categories

Every turn

GLiNER2 205M CPU Stable extraction against evolved schema — entities, topics, tones, decisions Every turn

Bonsai 8B ternary (~2.15 GB) GPU Relation extraction, query planning, anomaly verification Every turn + on wake

SSM (Mamba) ~500M GPU Working memory — continuous hidden state, no fixed token limit Real-time

JEPA Predictor ~300M GPU Saliency gate — predicts future retrieval need, triggers graph retrieval Real-time

GNN Consolidator ~200M GPU Saliency scoring, subgraph summarization, link prediction, anomaly detection, ontology refinement Dream state (offline)

Oracle (DeepSeek) Large Cloud API Training signal for all learned components Gestational training

3. Component Justifications

3.1 WaveDB Graph Layer → Hippocampal Index

Why not a vector database? Vector similarity answers "what's similar to this?" but not "what did Alice say about databases last week?" or "what happened after we implemented morphisms?" The graph stores **discrete, symbolic relationships** — entity presence, topic classification, affective state, temporal order, decisions, contradictions. These are different in kind from embedding similarity and answer different questions.

Why WaveDB specifically? WaveDB provides a triple store with SP0/POS/OSP indexes, Gremlin-style traversal, cost-based query optimization, and morphisms (reusable query fragments). The morphism system maps directly to **systems consolidation** — frequently traversed paths become single named pathways, analogous to how the neocortex learns direct connections after repeated hippocampal replay.

Key design decision: The graph stores **structure** (who, what, when, how felt, in what order). The HBTrie stores **content** (the actual words). The vector index stores **similarity**. Each answers different questions, and together they form a memory system no single component could provide.

3.2 WaveDB HBTrie → Neocortical Store

Why hierarchical key-value? The HBTrie's hierarchical paths (`episodes/ep_47/summary` , `entities/alice/properties/name`) map naturally to distributed cortical representations. Each level is a "cortical region" — same underlying storage, different organizational principles.

Why MVCC? Multi-Version Concurrency Control provides snapshot isolation — every read sees a consistent version. This maps to **state-dependent memory**: the same memory can have different "versions" depending on when you access it. Combined with edge validity windows in the Graph layer, this enables temporal queries ("what did I know at time T?") without losing historical versions.

3.3 GLiNER-Decoder → Open Entity Discovery

Why not just use a predefined schema? A static schema is a filter — it tells the system what to see, which means it also tells it what to ignore. The brain discovers categories through experience. GLiNER-Decoder (Knowledgator, 205M parameters) is a variant of GLiNER that uses a small generative decoder to **invent labels** for discovered entities rather than matching against a predefined list. Given `labels=["label"]` , it produces spans with freely generated type names like `"configuration option"` , `"deployment strategy"` , `"sentiment"` .

Why it matters: The system discovers that this user talks about `deployment` , `testing` , and `documentation` — categories no one predicted. These discovered labels are clustered, canonicalized, and promoted to the stable schema over time. The schema is alive.

3.4 GLiNER2 → Stable Extraction

Why two GLiNER variants? GLiNER-Decoder is flexible but inconsistent — the same entity might be labeled `"configuration option"` in one conversation and `"setting"` in another. GLiNER2 (Fastino, 205M parameters) extracts against a stable, evolved schema with consistent labels. It's faster and more reliable for categories that have been promoted from discovery.

Why 205M parameters? A 459M GLiNER encoder outperformed ChatGPT (~175B) on named entity recognition benchmarks. For extraction tasks, specialized small models beat generalist large models. GLiNER2 runs on CPU, costs nothing per inference, and produces structured output in a single forward pass.

3.5 Bonsai → Relation Reasoning, Query Planning, Verification

Why an 8B ternary model? Three distinct roles require reasoning beyond what GLiNER can do:

1. **Relation extraction:** GLiNER finds entities. Bonsai determines how they relate — "Alice explained DEBOUNCED," "User decided to use HBtrie," "This contradicts the earlier decision." This requires understanding the semantic content of the conversation, not just spotting named entities.
2. **Query planning:** Converting "What was I frustrated about last week?" into `{tones: ["frustrated"], temporal_filter: "last_week", entity_mode: "union"}` requires reasoning about query intent. GLiNER can extract entities from the prompt, but it can't determine that "Alice and I decided" means `entity_mode` should be "union" (episodes involving Alice) rather than "intersection" (episodes containing both Alice AND the user).
3. **Reality monitoring:** During consolidation, Bonsai wakes to verify GNN-flagged anomalies. The ternary weights $\{-1, 0, +1\}$ map to inhibitory/excitatory/absent synapses — the model can actively suppress incoherent reconstructions, not just fail to excite them. This is biologically closer to how the prefrontal cortex performs reality monitoring.

Why ternary? The 8B ternary model occupies ~2.15 GB — it fits on an RTX 5060 Ti (16 GB) alongside the SSM and JEPA. Ternary weights also map to biological synapse states: potentiated (+1), depressed (-1), or silent/absent (0).

3.6 SSM (Mamba) → Working Memory

Why not a transformer? Transformers have $O(n^2)$ attention — the thing we're trying to escape. State Space Models (Mamba) maintain a continuous hidden state that evolves with each input. There is no fixed context window. Old information decays gracefully rather than being truncated. The state dimension is constant, but the information it represents expands and contracts dynamically.

Why it's working memory: The SSM state IS the currently activated subset of everything the system knows. When the JEPA predictor determines the state is sufficient, no graph retrieval is needed — the SSM state alone generates the response. When JEPA detects low confidence (a novel entity, an unfamiliar topic), it triggers graph retrieval and the retrieved content is injected into the SSM state.

3.7 JEPA Predictor → Saliency Gate

Why not a rule-based gate? The decision of whether to retrieve from the graph or trust the SSM state depends on nuanced factors: how recently was this entity discussed? How confident is the SSM about this topic? Is this a follow-up question or a topic shift? A learned predictor (Joint Embedding Predictive Architecture) learns these patterns from experience.

Dual role:

- **Encoding gate:** Predicts future retrieval probability → modulates encoding strength. High-saliency conversations (decisions, strong emotions, novel entities) get stronger encoding.

Routine conversations get baseline encoding or are discarded.

- **Retrieval gate:** Predicts whether the SSM state covers the current prompt → triggers or skips graph retrieval. This is the efficiency mechanism — not every turn requires a graph traversal.

3.8 GNN Consolidator → Dream State Processing

Why a graph neural network? The consolidation tasks — salience scoring, subgraph summarization, link prediction, anomaly detection, ontology refinement — all require reasoning about **graph structure**, not just node features. A node's importance depends on its position in the graph (bridge, hub, peripheral), not just its content. GNNs are the standard tool for this class of problem.

Why offline? Consolidation is computationally intensive and benefits from seeing the full graph. Running it during the dream state (offline, periodic) mirrors the brain's sharp-wave ripple replay during sleep. The GNN identifies clusters for abstraction, predicts missing edges, detects anomalies, and refines the ontology — then Bonsai verifies the high-stakes decisions.

Specific GNN architectures:

- **Salience scoring:** Graph attention networks (GAT) learn to weight neighbor importance
- **Subgraph summarization:** DiffPool or MinCutPool layers collapse subgraphs into coarser representations
- **Link prediction:** Graph autoencoders (GAE) or SEAL predict missing edges
- **Anomaly detection:** Learned deviation from expected neighborhood patterns

3.9 Oracle (DeepSeek) → Gestational Training

Why an Oracle? Several components require training data that doesn't exist in public datasets: query planning pairs, relation extraction from conversational text, salience labels on memory graphs. The Oracle (a large model like DeepSeek) generates this data during a gestational training phase.

Why not permanent? The Oracle is expensive per call. It generates 5,000–10,000 training examples per component during initial training, then the components operate independently. The Oracle is only invoked again if the system encounters genuinely novel situations that the fine-tuned models can't handle.

4. Information Flow

4.1 Encoding Flow (Every Conversation Turn)

1. Raw conversation text enters the system

2. GLiNER-Decoder performs open discovery:
"WAL config" → "technical concept"
"DEBOUNCED" → "configuration option"
"confusing" → "sentiment"
"go with DEBOUNCED" → "decision"
→ Raw entity spans with invented type labels
3. GLiNER2 performs stable extraction against the evolved schema:
entities: ["Alice", "WaveDB"]
topics: ["configuration"]
tones: ["frustrated", "curious"]
decisions: ["use_debounced"]
→ Structured, consistent labels
4. Discovered labels are buffered. Labels appearing >10 times are clustered, canonicalized, and promoted to the stable schema.
"configuration option" + "setting" + "parameter" → "configuration"
5. Bonsai extracts relations:
(User, expresses, frustrated)
(User, questions, WAL_sync_modes)
(Alice, explains, WAL_sync_modes)
(User, decides, use_debounced)
(conversation, concerns, configuration)
(conversation, involves, Alice)
6. JEPA predicts encoding strength:
salience = 0.65 (moderate – contains a decision but routine topic)
→ Encode at baseline strength
7. Storage:
Graph layer: insert all triples as edges
HBtrie: store summary, full text, timestamp
Vector index: store embedding of summary for semantic search

4.2 Retrieval Flow (On User Prompt)

1. User prompt: "What was I frustrated about last week?"
2. SSM state absorbs the prompt. State evolves.

3. JEPA evaluates: "Does the SSM state cover this?"
 - "frustrated" tone not in recent state → low confidence
 - "last week" temporal scope → SSM state only covers recent turns→ TRIGGER GRAPH RETRIEVAL
4. GLiNER2 extracts entities/tones from prompt:
tones: ["frustrated"]
5. Bonsai plans the query:

```
{
  entities: [],
  topics: [],
  tones: ["frustrated"],
  entity_mode: "union",
  temporal_filter: "last_week",
  limit: 5
}
```
6. Graph traversal (pattern completion):
 - Find all episodes with tone "frustrated" (union)
 - Filter by timestamp > 7 days ago
 - Score by: tone match count + recency
 - Return top 5
7. HBtrie load (neocortical reactivation):
For each result episode ID:
load summary, full_text, timestamp from HBtrie
8. Working memory = activated graph nodes + HBtrie values + SSM state
9. Response generation from working memory

4.3 Consolidation Flow (Dream State – Periodic)

1. GNN scores all nodes/edges by structural salience:
 - ep_001: 0.92 (anchor episode, foundational decision)
 - ep_005: 0.30 (routine, redundant paths exist)
 - edge ep_003_has_entity_Bob: 0.80 (only connection for Bob)
2. GNN detects subgraphs ready for abstraction:
Cluster: [ep_001, ep_002, ep_003, ep_004]

- Abstract to: "WaveDB initial development (June 20-23)"
 - Store semantic memory in HBtrie
 - Link semantic memory to source episodes
 - Mark source episodes as "abstracted" (still retrievable)
3. GNN predicts missing edges:
- (Postgres, related_to, performance) confidence 0.82
 - Auto-accept (confidence > 0.8)
 - (ep_006, caused_by, ep_005) confidence 0.75
 - Propose for Bonsai verification (confidence 0.5-0.8)
4. GNN detects anomalies:
- MISSING_TEMPORAL: ep_007 follows ep_004, but ep_005-006 also follow ep_004
 - Wake Bonsai: "Are these parallel threads or should ep_007 follow ep_006?"
 - DUPLICATE_DECISION: use_hbtrie (ep_001) and use_wavedb_over_postgres (ep_003)
 - Wake Bonsai: "Are these the same decision?"
5. Bonsai verifies anomalies:
- Temporal branch: "Legitimate parallel threads. No change needed."
 - Duplicate decision: "Distinct decisions. use_hbtrie is about storage layer, use_wavedb_over_postgres is about database choice. Add 'reinforces' edge."
6. Pruning:
- Edges with utility_score < 0.1 and state="current" → state="archived"
 - Superseded edges older than 90 days → compress to summary, archive
 - Ontology categories not seen in 30+ days → deprecate
7. SSM replay:
- Replay high-saliency consolidated memories through SSM
 - Strengthens neocortical (SSM weight) representations

4.4 Forgetting Flow (Continuous + Periodic)

1. DECAY (continuous):
- Edge utility scores decay with disuse: $\text{score} *= (1 - \text{rate})^{\text{days_since_access}}$

- SSM state decays gradually (exponential moving average)
 - Discovered labels that stop appearing lose promotion eligibility
2. ACTIVE FORGETTING (user-triggered):
- User: "Forget what I said about Postgres"
 - Edge state → "deprecated"
 - `validity_end` → `now()`
 - Edge still exists, not returned by default queries
 - Retrievable via historical query: "what did I used to think?"
3. RECONSOLIDATION (on contradiction):
- New fact contradicts old fact
 - Old fact: state → "superseded", `validity_end` → `now()`
 - New fact: state → "current", `validity_start` → `now()`
 - Link: `new_fact` supersedes `old_fact`, `new_fact` contradicts `old_fact`
 - Both versions preserved. Default queries return current only.
4. PRUNING (periodic consolidation):
- Archived edges older than 365 days → deep archive
 - Superseded edges → compressed to summary in HBtrie
 - Nothing is ever truly deleted. Storage is cheap.
-

5. The Evolving Ontology

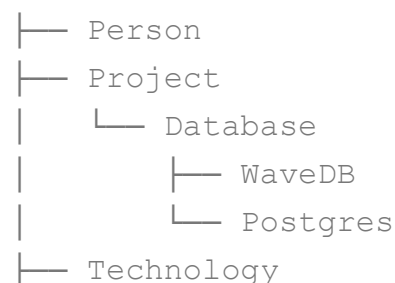
5.1 Why an Ontology?

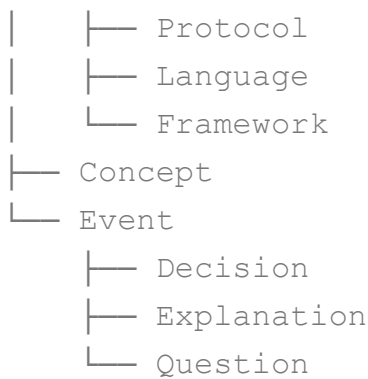
Without hierarchical knowledge, the system can't answer "What databases did we discuss?" — it would only match episodes explicitly labeled "database," missing episodes about WaveDB, Postgres, or SQLite. With a taxonomy where WaveDB IS-A Database, the query expands automatically.

5.2 How It Evolves

SEED ONTOLOGY (provided at initialization):

Entity





DISCOVERY (ongoing):

GLiNER-Decoder invents label "deployment strategy"

→ Buffered. After 10 occurrences:

→ Bonsai entailment: "Is 'deployment strategy' a Concept?"

→ Yes → added to ontology: Concept → DeploymentStrategy

REFINEMENT (periodic GNN pass):

GNN predicts: (DEBOUNCED, subclassOf, WALSyncMode) confidence 0.90

→ New class WALSyncMode created

→ DEBOUNCED, IMMEDIATE, ASYNC become subclasses

DECAY (periodic):

Category "Framework" not seen in 45 days

→ Deprecated. Entities reassigned to parent "Technology"

6. Implementation Plan

6.1 Phase 1: Bootstrap Extraction (Weeks 1-2)

Goal: All extraction components operational with acceptable accuracy.

Task Data Hardware Est. Time

GLiNER2 fine-tune on kniv-corpus-en 15K conversation examples with 18 NER types + 9 dialog act labels

RTX 4090 cloud (\$1.08) 4 hours

GLiNER-Decoder evaluation on Memory-Traces 27K human-LLM conversations with salience/emotion labels CPU (local) 2 hours

Bonsai relation extraction fine-tune Oracle-generated pairs (2K examples) A100 cloud (\$6.56) 4 hours

WaveDB schema setup Graph layer schema + seed ontology Local 1 day

Deliverable: System can encode conversations into structured triples with entities, topics, tones, decisions, and relations.

6.2 Phase 2: Train Memory Dynamics (Weeks 3-4)

Goal: JEPA, SSM, and Bonsai query planning operational.

Task Data Hardware Est. Time

JEPA salience training Memory-Traces labels (landmark, emotional_intensity, open_threads) RTX 4090 cloud (\$6.48) 24 hours

SSM pre-training DialogSum + SAMSum (30K dialogues) A100 cloud (\$61.92) 48 hours

Bonsai query planning Oracle-generated pairs (5K examples) A100 cloud (\$16.40) 10 hours

Integration testing Manual conversation scenarios Local 2 days

Deliverable: System can retrieve relevant memories given natural language questions, with JEPA gating retrieval.

6.3 Phase 3: GNN Consolidator (Weeks 5-6)

Goal: Dream state consolidation operational.

Task Data Hardware Est. Time

GNN architecture implementation GAT + DiffPool + GAE RTX 4090 cloud (\$4.32) 16 hours

Initial GNN training Oracle-generated salience labels + link prediction examples RTX 4090 cloud (\$8.64) 32 hours

Consolidation loop integration WaveDB graph → GNN → Bonsai verify → WaveDB update Local 3 days

Deliverable: Nightly consolidation pass that scores salience, abstracts subgraphs, predicts links, and detects anomalies.

6.4 Phase 4: Evaluation & Iteration (Weeks 7-8)

Goal: Quantify performance against benchmarks.

Benchmark What It Tests Target

ConvoMem 75K QA pairs, 6 evidence categories, 100 personas >80% recall on single-session, >60% on cross-session

EverMemBench Multi-party, 250-day span, 1M+ tokens >40% on multi-hop (baseline is 26% with oracle evidence)

CloneMem 1-3 year temporal reasoning >50% on temporal comparison

6.5 Phase 5: Self-Generated Training (Ongoing)

Goal: System improves from its own operation.

- Retrieval successes → positive salience signal for edges used

- Retrieval failures → training examples for missing edges
 - Bonsai anomaly verifications → training examples for GNN anomaly detector
 - User corrections → reconsolidation training examples
 - New entity types discovered → ontology refinement examples
-

7. Cost Analysis

7.1 Cloud Training Costs

Phase Component GPU Provider Hours Cost

1 GLiNER2 fine-tune RTX 4090 Vast.ai spot 4 \$1.08

1 Bonsai relations (Oracle gen) A100 80GB RunPod Community 4 \$6.56

2 JEPA salience RTX 4090 Vast.ai spot 24 \$6.48

2 SSM pre-training A100 80GB Lambda Labs 48 \$61.92

2 Bonsai query planning A100 80GB RunPod Community 10 \$16.40

3 GNN implementation RTX 4090 Vast.ai spot 16 \$4.32

3 GNN training RTX 4090 Vast.ai spot 32 \$8.64

TOTAL 138 \$105.40

7.2 Oracle API Costs

Use Tokens (est.) Cost (DeepSeek)

Generate 5K query planning pairs ~~2.5M input + 1M output → \$3.50~~

~~Generate 2K relation extraction examples 1M input + 0.5M output → \$1.75~~

~~Generate 1K salience labels 0.5M input + 0.3M output → \$1.00~~

~~Generate 500 ontology refinement examples 0.3M input + 0.2M output → \$0.70~~

TOTAL **\$7.00**

7.3 Local Hardware (Already Available)

Component Hardware Status

GLiNER2 inference CPU  Runs on any modern CPU

GLiNER-Decoder inference CPU  Runs on any modern CPU

Bonsai inference (8B ternary) RTX 5060 Ti (16 GB)  ~~2.15 GB fits comfortably~~

SSM inference (500M) RTX 5060 Ti (16 GB)  Fits in 16 GB

JEPA inference (300M) RTX 5060 Ti (16 GB)  Fits in 16 GB

GNN inference (200M) RTX 5060 Ti (16 GB)  Fits in 16 GB

WaveDB CPU/disk  Production-ready

7.4 Total Cost Summary

Category Cost

Cloud GPU training (all phases) \$105

~~Oracle API (DeepSeek) ~\$7~~

~~Local hardware \$0 (already owned)~~

~~WaveDB Free (MIT license, self-hosted)~~

~~**GRAND TOTAL** **\$112**~~

8. Key Design Decisions & Tradeoffs

8.1 Why Not a Single Large Model?

A single large model that does extraction, retrieval, and generation would be simpler. But:

- **Cost:** Running a 70B+ model for every conversation turn is expensive. GLiNER2 runs on CPU and costs nothing per inference.
- **Specialization:** A 459M GLiNER encoder outperformed ChatGPT (~175B) on NER. Specialized small models beat generalist large models on their specific tasks.
- **Interpretability:** When retrieval fails, you can inspect the graph traversal to see why. A large model's retrieval is a black box.
- **Updateability:** You can fine-tune GLiNER2 on new entity types without retraining the entire system.

8.2 Why Not Pure Vector Search?

Vector search answers "what's similar to this embedding?" It doesn't answer:

- "What did Alice say about databases?" (entity + topic constraint)
- "What happened after we implemented morphisms?" (temporal chain)
- "What decisions did we make and why?" (relation type constraint)
- "What was I frustrated about?" (affective state constraint)

The graph stores these as discrete, queryable relationships. Vector search is complementary — it handles the "similar to this" queries that the graph can't.

8.3 Why Offline Consolidation?

Running the GNN on every conversation turn would add latency and computational cost. The brain doesn't consolidate in real-time either — it replays memories during sleep. The offline "dream state" pass

allows the GNN to see the full graph, make global decisions about salience and abstraction, and not compete with real-time inference for GPU resources.

8.4 Why Never Delete?

Storage is cheap. The value of being able to answer "what did I used to think?" or "what changed and when?" outweighs the cost of keeping superseded edges. MVCC in WaveDB already versions every write. Edge validity windows add temporal semantics. The combination gives you current, historical, and change views without destructive operations.

9. Success Metrics

Metric Target Measurement

Encoding accuracy >90% entity F1, >85% relation F1 Against Oracle-generated ground truth

Retrieval recall >80% on ConvoMem single-session, >60% cross-session Standard benchmark evaluation

Retrieval precision >85% Retrieved episodes contain answer

Retrieval latency <50ms for graph traversal + HBTrie load WaveDB benchmarks show 2.6M reads/sec

****Consolidation quality**** >70% of GNN-predicted edges validated by Bonsai Bonsai accept rate on proposed edges

Forgetting accuracy <5% of pruned edges later needed Track retrieval failures on archived edges

Context efficiency Graph-based context $\leq 50\%$ the size of full-history context for equivalent recall Character count comparison

10. References

- Teyler, T.J. & DiScenna, P. (1986). The hippocampal memory indexing theory. *Behavioral Neuroscience*, 100(2), 147-154.
- Cowan, N. (1999). An embedded-processes model of working memory. In A. Miyake & P. Shah (Eds.), *Models of Working Memory*.
- McClelland, J.L., McNaughton, B.L., & O'Reilly, R.C. (1995). Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3), 419-457.
- WaveDB: Hierarchical key-value database with MVCC and Graph layer.
github.com/vijayee/WaveDB
- GLiNER2: Fastino AI. Zero-shot NER + classification + relation extraction. github.com/fastino-ai/GLiNER2
- GLiNER-Decoder: Knowledgator. Open-ontology entity extraction.
huggingface.co/knowledgator/gliner-decoder-base-v1.0

- Memory-Traces: Conversational memory dataset with salience, emotion, and identity labels.
huggingface.co/datasets/Cossale/memory-traces
 - ConvoMem: Salesforce. Long-term conversational memory benchmark.
github.com/SalesforceAIResearch/ConvoMem
 - EverMemBench: Multi-party, long-horizon memory benchmark.
huggingface.co/datasets/EverMind-AI/EverMemBench-Dynamic
 - CloneMem: Temporal reasoning over years of digital traces.
huggingface.co/datasets/ZhiyuZhangA/CloneMem
-

This architecture is designed to be built incrementally. Each phase produces a working system. Phase 1 alone – GLiNER2 extraction + WaveDB storage + basic graph retrieval – is a functional memory system that outperforms sliding-window context. Each subsequent phase adds capability without requiring redesign of earlier components.